MessageFoundry — System Requirements

Early Access · MessageFoundry v0.3.2 · July 2026

These are the minimum and recommended requirements for running the MessageFoundry (MEFOR) engine, its message store, and the administration clients. The engine is a headless Python/asyncio service; the operator console is a browser page served by the engine itself at `/ui`.

> **On the throughput figures.** MEFOR **does** publish a measured per-node throughput baseline: `benchmarks/TUNING-BASELINE.md` is **canonical** for every measured figure, with THROUGHPUT.md as the plain-language guide to reading it. What it publishes is a reproducible **method** plus numbers stamped "as measured on the reference config" — not a headline capacity number, because the durable-write path makes throughput hardware-dependent. The sizing tiers in Sizing by message volume are **derived from those published measurements**: every tier's peak rate traces to a named measured run, and its daily figure is that rate through a **stated duty cycle**, not `× 86,400`. They are still not guarantees — each one inherits its source run's hardware, **fan-out** and transform cost. Always establish your own baseline on production-like hardware before go-live (see LOAD-TESTING.md, the synthetic load-test profiles in `harness/load/`, and § Capacity notes).

Hardware

	Minimum (lab / low-volume pilot)	Recommended (single-node production)
CPU	2 cores	4+ cores (transform throughput is per-core; one worker set per connection)
Memory	4 GB	8–16 GB
Disk	10 GB free, any disk	**SSD**, 50+ GB or sized to your retention window, on a low-latency local volume
Store volume	—	Put the message store on a fast local disk (not a network share). Budget for **store + WAL growth**: the staged pipeline writes ~3× per message on the embedded store (see write-amplification benchmark).

> The engine and store may share a host for a low-volume pilot. For production, run a **server database** (PostgreSQL or SQL Server) on its own host, sized by your DBA, and keep the engine host dedicated. For volume beyond one CPU core, see Sizing by message volume.

Hardware memory encryption — required for an ASVS Level 3 PHI deployment

Requirement. A deployment claiming ASVS **Level 3** for PHI **must** run the engine on a host that provides **full memory encryption** — **AMD SEV-SNP** (EPYC 7003 "Milan" or later) or **Intel TDX** (5th Gen Xeon Scalable or later) — *and* must actually launch the engine's VM as a **confidential guest** on that platform. Capable silicon that is not running the guest in confidential mode does not satisfy it. This is ASVS 11.7.1 (*full memory encryption … protects sensitive data while it is in use*), and it exists because PHI is **plaintext in process memory** for as long as the engine is parsing, routing and transforming it — HL7 is `str` end to end, and no application-level control can encrypt the interpreter's heap.

This is a procurement decision, and we say so plainly. MessageFoundry cannot provide the property; only the host can. On a host that does not provide it:

- the engine **still runs** — nothing here is a functional requirement, and every other PHI control (at-rest encryption, retention, audit, RBAC, transport) is unaffected;
- ASVS 11.7.1 is capped at **Partial**, not Pass, and that cap is a **hardware fact about your deployment**, not a gap in the software. Disclose it in your own assessment rather than working around it;
- an **exposed** PHI instance **warns at every start** until the decision is recorded — `[security].memory_encryption_operator_declared = true` is the operator's declaration that the host provides it. The engine **starts either way**: this is a host property, not a config error, and one no operator can satisfy on Windows, so refusing by default would break deployments over something they cannot change. An estate that has standardized on confidential-computing hosts can make the missing declaration fatal with `[security].require_memory_encryption_declaration = true` (default `false`). Loopback and synthetic instances are unaffected and silent. See CONFIGURATION.md `[security]` and OFF-LOOPBACK-DEPLOYMENT.md.

What the engine does and does not tell you. `GET /security/posture` carries a **report-only** platform read-out (`memory_encryption_self_reported_capability` / `..._self_reported_active` / `..._self_reported_mechanism` / `memory_encryption_readout_source`). On Linux it reads `/proc/cpuinfo` flags for *capability* and `/dev/sev-guest` / `/dev/tdx_guest` presence for *activation*; on **Windows every field is `null`** (see below). **No value of any of those fields satisfies 11.7.1** — they are values the host OS emits about itself, and 11.7.1 exists precisely because that host may be the adversary. Only a CPU-signed attestation report verified against the silicon vendor's root PKI would be evidence, and **that is not built**. The response states this itself in `memory_encryption_note`, so the limitation travels with any copy of the posture body. The engine measures and reports; your deployment determines the verdict.

Availability on today's on-premises platforms (verified 2026-07-22). For a **Windows** engine host — the primary supported platform — this requirement is presently **unmeetable on-premises**, and the blocker is the hypervisor, not the CPU:

Platform	Confidential guest for a engine VM
Hyper-V on-premises (Windows Server 2019–2025)	⛔ None. Windows Server 2025 ships no confidential VM; the vNext Insider "Trusted Launch" (Secure Boot + vTPM) is **not** memory encryption.

Platform	Confidential guest for a engine VM
VMware ESXi 9.0	⚠️ SEV-SNP is a *Limited Availability* release and its guest requirements are stated in Linux-kernel terms; Windows is not listed as a supported SEV-SNP guest.
Azure / Azure Local confidential VMs	✅ SEV-SNP and TDX with Windows Server guests (a Microsoft paravisor supplies what a Windows guest needs).
Linux guests on KVM / AWS / GCP / ESXi 9	✅ Where the host is SEV-SNP/TDX-capable and the guest is launched confidential.

So a Windows on-prem deployment is honestly capped at **Partial** today, and a Linux or Azure confidential-VM deployment is the route to the hardware property. Plan hardware refresh accordingly — SEV-SNP needs EPYC 7003+ and TDX needs 5th Gen Xeon Scalable+, which is newer than much of a typical 5–7-year hospital server refresh cycle.

Operating systems

Platform	Status
Windows Server 2022 / 2025	✅ Primary supported & serviced platform (Windows-service deployment via NSSM)
Windows Server 2019	✅ Supported
Windows 10 / 11	✅ Supported (development, pilot, test-harness host)
Linux (modern x86-64 distributions)	✅ Engine supported (cross-platform Python); no bundled service installer — run under systemd yourself
macOS	⚠️ Development / test-harness use only

Runtime

Component	Requirement
Python	**3.14**, 64-bit (the only supported runtime; CI-validated on Linux + Windows Server 2022 + 2025, the primary deploy target)
Service manager (Windows)	**NSSM** (auto-provisioned, SHA-256-pinned, by the installer; or pre-staged). Requires administrator / elevation to register the service.
C compiler	Not required for the default install (runtime dependencies ship as wheels)

Databases (message store)

Database	Status	Driver / prerequisite
SQLite (WAL)	✅ Default, bundled — single-node	None (`aiosqlite` , in-process)
PostgreSQL 13+	✅ Production	`messagefoundry[postgres]` extra (`asyncpg` — no OS dependency; ships compiled wheels)
Microsoft SQL Server 2022 / 2025	✅ Production	`messagefoundry[sqlserver]` extra (`aioodbc`) **plus the OS-level Microsoft ODBC Driver 18 for SQL Server** (18.5+ covers both majors). Read-Committed Snapshot Isolation (RCSI) recommended. SQL Server 2025 requires an AVX-capable CPU.
MySQL / Oracle	⛔ Not supported (roadmap)	—

> The embedded SQLite store needs no setup and suits pilots and single-node deployments. A **server database is greenfield-only** — there is no in-place migration from a populated SQLite store; drain and cut over. The DB tier owns its own backup, HA, and (SQL Server) TDE / purge maintenance. A server database is also the **concurrency / scale substrate** — see below.

Administration clients

Client	Requirement
Web console (the operator UI)	A modern browser — **nothing to install on the operator's machine.** The engine serves the console same-origin under `/ui` from its own FastAPI app (ADR 0065), **on by default** since ADR 0143 (`[security].serve_web_console` ; set it to `false` for a JSON-API-only deployment). It ships as a separately-versioned wheel, `messagefoundry-webconsole` , mounted in-process. This is the **sole operator console** — the PySide6 desktop console was retired (ADR 0032).
VS Code extension	Visual Studio Code (current stable) — route wizard, validate-on-save, test bench, stage→promote.
Test harness — *optional, not needed to run the engine*	The standalone synthetic send/receive/load harness (`python -m harness`) is the **only** PySide6 (Qt) surface left: `pip install messagefoundry[harness]` , Windows / Linux / macOS, a separate process reaching the engine only over the HTTP API. It is a **testing tool** — an engine host that does not run it needs no Qt and no GUI at all.

Network & ports

Purpose	Default	Notes
Engine API (HTTP + WebSocket)	`127.0.0.1:8765`	**Loopback by default**, authentication-required. **In-process TLS is built and opt-in** (WP-13a, ADR 0002): set `[api].tls_cert_file` (plus `tls_key_file` when the key is a separate PEM) and the engine terminates TLS in uvicorn, so the API **and** the `/ws/stats` WebSocket serve `https` / `wss`. TLS **1.2 floor** (`tls_min_version` — `1.2` or `1.3`), optional `tls_ciphers`, and **opt-in mTLS** via `tls_client_ca_file` (a client certificate is then required and verified). A **TLS-terminating reverse proxy** remains the supported alternative (`tls_terminated_upstream` + `trusted_proxies`). An off-loopback bind needs one of the two — in-process TLS needs no override flag, and the browser console refuses an unprotected off-loopback bind outright.
Inbound MLLP / TCP listeners	operator-defined (samples use e.g. `2575`, `2600`)	Open to sending systems via firewall. **MLLP-over-TLS is built and opt-in per connection** (WP-13b, `tls = true`, TLS 1.2+ — see CONNECTIONS.md): an inbound presents `tls_cert_file` / `tls_key_file` as its server identity and opts into **mTLS** with `tls_ca_file`; an outbound **verifies the partner's certificate by default** (`tls_verify`, `tls_check_hostname`, both `true`). Plaintext is still the **default**, so a non-loopback MLLP listener should set `tls = true` — and a cleartext MLLP **egress** off loopback is **refused at startup** on a production-PHI instance unless the hop is attested.
Outbound	as configured	Reachability to downstream partners and, for server DBs, to the database host.
Installer egress	HTTPS	Outbound access for the service installer to fetch the pinned NSSM binary (or pre-stage it).

Sizing by message volume

> **Derived from measurement — still not a committed number.** Each tier's peak rate below is a rate we have actually measured, named and sourced under *Reading the tiers*, and **no tier projects above a measured rate**. They are still not guarantees: every one carries its source run's hardware, **fan-out** and transform cost, and throughput depends heavily on **transform cost per message** (the dominant factor), message size, fan-out, and strict-validation use. **Measure your own feeds** with the load harness before committing (see Capacity notes).
>
> The measured figures — and the exact conditions they were measured under — live in `benchmarks/TUNING-BASELINE.md` (canonical) and THROUGHPUT.md; the anchors are restated with their arithmetic under *Reading the tiers* below. A tier row is a **sizing starting point**, never a demonstrated capability claim.

How throughput is bounded (read this first)

A single engine process runs **all** message work — decode → peek → route → transform → re-encode — on **one CPU core** (one asyncio event loop; the GIL prevents pure-Python parallelism across threads). So per-process throughput is governed, in order, by:

1. **Transform cost per message** — usually the binding constraint. The project's own throughput research cites a comparable vendor benchmark where real transformation cut pass-through throughput by ~60% (≈1000 msg/s → ≈400 msg/s). A light/pass-through feed sits near the top of a tier; a heavy transform sits near the bottom.
2. **Durable-write cost** — every stage handoff (ingress → routed → outbound → delivered) is a committed transaction. In-process **SQLite is fastest per write**; a **server DB is slower per single write** (network + MVCC) but is the concurrency substrate (next point).

To exceed one core today, scale **intra-node** on a server DB: many connections / lanes / delivery workers draining **one shared server database** (PostgreSQL or SQL Server) concurrently via `SELECT ... FOR UPDATE SKIP LOCKED` + row leases. Throughput scales with workers until the **database's commit capacity** is the wall. (SQLite is single-writer and does **not** scale this way — it is the single-process / single-node store.) Engine HA is **single-leader active-passive** — the graph runs on the leader only. A **multi-process, sharded-by-inbound** scale-out (multiple engines, each owning a disjoint set of inbounds) **is built** — `messagefoundry supervise` (ADR 0037); with more than one shard it **requires a server DB** so all shards share **one unified store** (ADR 0063), and the N-concurrently-active reliability runtime is built by ADR 0073: startup/DR crash recovery is **ownership-scoped** (a restarting shard re-pends only its own lanes' in-flight rows, never a live sibling's) and each outbound lane has a **single delivery consumer** (deterministic rendezvous ownership, so per-lane FIFO holds across shards). Sharding and `[cluster]` active-passive are mutually exclusive (refused at startup); a shard-set change requires a coordinated fleet restart (reload refuses it). **Certification status:** the mechanism is built and invariant-tested, but N-active on one store is not yet certified as a supported production topology — that flips only after the clean 4-engine no-loss bench (sustained, zero loss, per-lane FIFO). Until then, treat multi-engine deployments as **active-passive** (one active writer per store) for production sizing.

> **Connection-count guidance.** On a server-DB store, the pre-ADR-0066 `per_lane` topology ran a claim loop per connection per stage against the shared queue; at very high connection counts the *store's* claim path saturated on lock contention **independent of message volume** (measured: ~1,500 connections pinned an 8-vCPU store box at idle). The **default `pooled` claim mode** (ADR 0066, the default since #744) **collapses that claim storm** — a handful of shared per-stage claimers (`StageDispatcher`) replace the ~1,500 loops — so at high connection counts, keep the default `pooled`. If you pin `[pipeline].claim_mode = "per_lane"`, size deployments to **no more than a few hundred connections per store** and enable `[pipeline].per_lane_wake` so idle connections cost the store ~nothing. Two caveats of running at the scale pooled unlocks — **exactly-once degrades under load** (no inbound de-duplication, so receivers must be idempotent) and the flip evidence is **single-node** (failover duplicate/ordering paths unmeasured; the T17 infra-fault limitation is tracked by ADR 0070) — are documented in the "Pipeline claim mode" section of CONNECTIONS.md.

Tiers

Tier	Peak sustained ingress	Indicative daily volume	Deployment shape	Store	Suggested hardware (engine host)
Pilot / light	up to ~30 msg/s	up to ~1.0 M/day	1 process, single node	SQLite	2 cores / 4 GB
Standard single-node	~30–70 msg/s	~1.0–2.2 M/day	1 process, single node	SQLite, or PostgreSQL / SQL Server	4 cores / 8 GB
High single-node	~70–100 msg/s	~2.2–3.2 M/day	1 process, tuned (lean transforms; low fan-out; the default `pooled` claim mode; finite-retry on hot lanes), many connections / lanes draining concurrently via `SKIP LOCKED`	Server DB on its **own** host (PostgreSQL / SQL Server)	4–8 cores / 16 GB + a DB host sized to the commit load
Multi-process (engine shards)	~165 msg/s measured at 4 engine shards (**per-shard SQLite**); beyond that ≈ *N* × your measured single-shard rate × 0.85	~5.3 M/day at that measured rate	*N* `messagefoundry supervise` engine-shard processes, partitioned by inbound connection — **not yet certified as a production topology** (see above)	**PostgreSQL / SQL Server** — required above one shard, so all engine shards share **one unified store**	8+ cores / 32 GB + a dedicated DB host sized to the commit load

Reading the tiers — and checking the arithmetic

Every peak figure in the table is a **measured** rate with a named source run — the only extrapolation is the per-shard multiplier on the last row. Where a tier's *hardware* row was never itself measured, its bullet says so rather than interpolating. Here is the derivation, so you can check it.

- **Units.** *Peak sustained ingress* is **messages received per second**, at the **low fan-out (~1–2 destinations per received message)** of every anchor run below. A high-fan-out hub commits and delivers several copies per received message and sustains a **much lower** ingress rate — see the last bullet.
- **Daily = peak × 86,400 ÷ 2.7**, i.e. `peak × ~32,000` — **not** `peak × 86,400`. Healthcare feeds are bursty: in the production ADT feeds profiled in THROUGHPUT.md §6 the **busiest hour runs ~2.7× the all-day average**, and you must size to the peak hour, so a peak-rate figure carries only `86,400 / 2.7` seconds' worth of messages a day. That is the duty cycle assumed in the *indicative daily volume* column, and it is the same arithmetic THROUGHPUT.md §6 works through (60 msg/s → ~1.9 M/day, not ~5.2 M). Substitute your own peak-hour ÷ daily-average ratio once you have measured one.
- **Pilot / light — ~30 msg/s** is the **lowest** sustainable rate measured across the three backends on the published reference config: **~30 msg/s on SQL Server**, conformance-clean, fan-out 2, on a 4-vCPU runner with the database co-located (TUNING-BASELINE.md §Results). **No 2-core point has ever been measured**, so

this tier deliberately takes the floor of the measured band rather than interpolating a figure for its own hardware row. `30 × 32,000 ≈ 0.96 M/day`.

- **Standard single-node — ~30–70 msg/s** is that same reference config's full measured band: **~30 (SQL Server) · ~50 (PostgreSQL) · ≥ 70 (SQLite, still not saturated at the top rate step)**. It brackets the **~60 msg/s end-to-end** ceiling measured for one strictly-ordered interface against an *instant-acknowledging* partner, versus **~450 msg/s at intake** (ACK-on-receipt) — i.e. **~16 ms** of engine overhead per message (THROUGHPUT.md §8). `30–70 × 32,000 ≈ 0.96–2.24 M/day`. The ~1000 msg/s pass-through figure quoted above is a **vendor's** self-benchmark, not a MEFOR measurement.
- **High single-node — ~70–100 msg/s** is the highest rate ever measured from **one engine process**: **~97 msg/s max sustainable** (zero-loss, bounded backlog) and **~107 msg/s** as a burst that still drained — one process, **1,500 inbound connections**, the default `pooled` claim mode, SQL Server 2022 on its own box, fan-out 1 (`benchmarks/adr0066-pooled-claimer-744.md` §2). Engine CPU sat at ~2 cores throughout and a 3× larger store pool did not move the ceiling, so this is the **ceiling of a single engine process** — the tier above it is not "more concurrency", it is **more processes**. `70–100 × 32,000 ≈ 2.24–3.2 M/day`.
- **Multi-process (engine shards) — ~165 msg/s at 4 shards.** Measured 1 → 2 → 4 engine shards at **~50 → 88.7 → 165.5 msg/s** aggregate, i.e. ~linear scaling at an efficiency **η ≈ 0.85** per added shard (TUNING-BASELINE.md §Multi-process sharding scale-out). `165.5 × 32,000 ≈ 5.3 M/day`. **Two limits travel with this row.** That run used **per-shard SQLite on a consumer 8-core test box**, so the portable result is the **speedup shape**, not the absolute rate: multiply η by *your* measured single-shard rate. And a production multi-shard deployment must share **one unified server-DB store** (ADR 0063) — a shape that run never exercised, and one measured **worse** (next bullet).
- **Where the tiers stop, and why.** Fan-out and the shared store — not the tier label — set the real number. On **one shared SQL Server store at a fan-out of 8**, the definitive 900-second soak of a 4-shard fleet sustained **10 msg/s of ingress** (80 deliveries/s, 90 total message events/s), and a fixed light load scaled cleanly only to **4 engine shards** — N = 8 and N = 16 collapsed (`benchmarks/THROUGHPUT-STATUS-2026-07-10.md` §3). Nor do per-interface ceilings **add**: a measured 16-lane run delivered **87/s in aggregate — 5.44/s per lane** — where summing the per-lane ceilings would have predicted ~960/s, an **~11× over-report** (THROUGHPUT.md §7). Always take `min(measured concurrent aggregate, Σ per-interface)`, and measure a high-fan-out hub rather than sizing it off the low-fan-out rows above. **~165 msg/s of ingress is the highest end-to-end rate measured anywhere in this project**; no figure in this document may be quoted as more.
- **Do not size on a future per-core lever.** Group-commit and the wider "reduce committed transactions per event" lever are **closed, not pending** — group-commit was withdrawn (ADR 0055) and the pre-registered measurement returned ABANDON at an elasticity of −0.115 (ADR 0107).

> **Single-stream server-DB caveat.** Because each staged handoff is a committed round-trip, a single delivery worker against a server DB drains far slower than in-process SQLite (the published baseline measures **~30 msg/s sustainable on SQL Server** vs **≥ 70 on SQLite** for one stream — TUNING-BASELINE.md §Results). High volume on a server DB comes from **concurrency** — many connections / lanes / processes draining in parallel — not single-stream speed. Size the DB host for that concurrent commit load.

Capacity notes

- Validate with the load harness (LOAD-TESTING.md): run the `smoke` → `fanout-baseline` → `soak` ramp, exercise the `cheap` / `edit` / `slow` transform modes to find your per-core transform ceiling, and compare SQLite vs a server DB on identical traffic. Treat the **zero-loss reconciliation** as the headline gate — throughput is meaningless if messages were lost.
- The embedded store has ~3× write amplification and a single-writer ceiling; move to PostgreSQL or SQL Server when that becomes the bottleneck.
- Scale **intra-node** on a server DB (one delivery worker per outbound; many connections / lanes draining concurrently via `SKIP LOCKED`; keep retry policies finite where head-of-line blocking on a shared FIFO lane would otherwise stall a lane). A multi-process **engine-shard** scale-out beyond one engine **is built** (`messagefoundry supervise` — ADR 0037, ADR 0063, ADR 0073); it needs a server DB so all shards share one unified store, and it is **not yet certified as a production topology** — see Sizing by message volume. Engine **HA** is **active-passive failover** (opt-in leader/standby cluster on shared PostgreSQL — see CLUSTERING.md); delegate **DB-tier** HA to the database + a load-balancer VIP.
